Session 3 – Gibbs posteriors, hold-out, imbalance and boosting

Goal. By the end of the session you should be able to build voters so that the prior is legitimate, compare self-bounding and hold-out certificates, certify the balanced error, and understand why boosted votes are hard to certify.
To reread before the session. Sections 4, 6 and 7.3 of the notes; notebooks 2 and 4.
Timing (3 hours).

	Part 3.1	50 min
	Part 3.2	35 min
	Part 3.3	50 min
	Part 3.4	45 min

To hand in at the end of the session. Your notebook, with the figure of Part 3.1 and the tables of Parts 3.2 to 3.4.

Part 3.1 – The Gibbs posterior and its temperature (Section 4)

We now build the voters ourselves, so that the prior can be fixed before seeing the data used in the bound.

1. On `wdbc` and `australian`, keep 20% of the data to build the voters: all the decision stumps $x \mapsto s\,\mathrm{sign}(x_j - \theta)$, $s \in \{\pm 1\}$, whose thresholds θ are the quantiles $10, 25, 50, 75, 90\%$ of feature j on this part. Split the remaining data into a sample S (60%) used by the bound and a test sample.

2. With the uniform prior, compute the Gibbs posterior $Q_\lambda \propto e^{-\lambda m \widehat{R}_S(h)}$ for a grid of 60 values of λ between 10^{-3} and $10^{1.5}$, and plot its empirical Gibbs risk, $\mathrm{KL}(Q_\lambda \| P)/m$ and Seeger's bound as functions of λ.

3. Report the best certificate, the corresponding test Gibbs risk and test risk of the vote, and compare with the Occam bound of the stump with the smallest empirical risk. Why is choosing λ on the grid free?

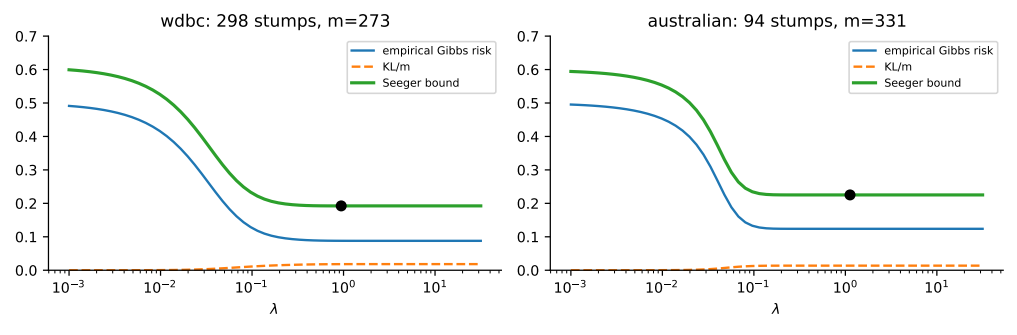
> **Numerical tip.** Compute the Gibbs posterior with a stable softmax, $Q_\lambda = \mathrm{softmax}(\ln P - \lambda m \widehat{R}_S)$, which subtracts the maximum before exponentiating; the naive formula underflows as soon as λm is large.

Solution

Algorithm 9: Gibbs posterior on stumps and choice of the temperature

split the data into S_{prior} (20%), S and a test sample;
$\mathcal{H} \leftarrow$ stumps with thresholds at the quantiles of each feature on S_{prior}; $P \leftarrow$ uniform on $\mathcal{H}$;
$\widehat{R}_S(h) \leftarrow$ error of each stump on S ($m = |S|$);
for λ *in a geometric grid* **do**
  $Q_\lambda \leftarrow \mathrm{softmax}(\ln P - \lambda m \widehat{R}_S)$;
  $B(\lambda) \leftarrow \overline{\mathrm{kl}}^{-1}(Q_\lambda^\top \widehat{R}_S, \frac{\mathrm{KL}(Q_\lambda \| P) + \ln(2\sqrt{m}/\delta)}{m})$;
return $\lambda^\star = \arg\min B(\lambda)$, $Q_{\lambda^\star}$, $B(\lambda^\star)$; ▷ *free:* λ *indexes posteriors*

Python: function `stumps, stump_votes, part4` of `tp_solutions.py`.



				best Gibbs posterior					ERM stump (Occam)	
dataset	n	m	λ	$\widehat{R}_S(G)$	KL	Seeger	test Gibbs	test vote	certificate	test
wdbc	298	273	0.943	0.088	4.98	0.192	0.104	0.104	0.196	0.104
australian	94	331	1.124	0.124	4.54	0.225	0.158	0.158	0.225	0.158

Shape of the curves. For small λ the posterior is the prior; since every stump comes with its opposite, the empirical Gibbs risk is exactly 0.5 and the bound is vacuous. When λ grows the empirical risk drops quickly while KL/m grows slowly, up to $\ln(n)/m$, and the bound reaches a minimum. Beyond this point the KL keeps increasing while the empirical risk no longer decreases.
Numbers. The best certificates are 0.192 (`wdbc`) and 0.225 (`australian`), for test Gibbs risks of 0.104 and 0.158. They are obtained for $\lambda \approx 1$, where the posterior is already concentrated on a few stumps (KL = 4.98 and 4.54 nats, close to $\ln n = 5.70$ and 4.54): on these problems a single feature is very informative, and the best Gibbs posterior is nearly the ERM stump, so its certificate is essentially the Occam bound (0.196 and 0.225) and its vote is essentially that stump. The Gibbs posterior does better than Occam when several voters are about equally good, as with the 300 stumps of `breast cancer` in Section 4 of the notes.
Why is the grid free? λ only indexes posteriors: Seeger's bound holds simultaneously for all Q, hence for all Q_λ, whatever the way λ is chosen (Theorem 6.2). No union bound is needed, contrary to the parameter C of Catoni's bound (Section 6.3).

Part 3.2 – Hold-out certificate versus self-bounding certificate (Section 6.4)

1. *Self-bounding:* learn a forest on the whole training set, minimize its tandem bound, and report its TND certificate. *Hold-out:* learn the same forest on 70% of the training set and compute the kl test-set bound of its (uniform) vote on the remaining 30%. Compare the test risks and the certificates on `spambase`, `satimage`, `pageblocks`, `abalone8`, `wdbc`, `german`.
2. Which approach gives the better vote? the better certificate? Explain the gap between the two certificates.
3. In which situation would you nevertheless recommend the self-bounding approach?

Solution

Algorithm 10: Hold-out versus self-bounding

; ▷ *self-bounding: all the training data*
forest on S; $\rho \leftarrow$ `optimize_tnd`; certificate $\leftarrow$ TND certificate of ρ (OOB);
; ▷ *hold-out*
split $S = S_1 \cup S_2$ (70/30); forest on S_1; $\hat{r} \leftarrow$ error of its vote on S_2;
certificate $\leftarrow \overline{\mathrm{kl}}^{-1}(\hat{r}, \ln(1/\delta)/|S_2|)$;

Python: function `partC` of `tp_solutions.py`.

dataset	m	self-bounding test risk	self-bounding certificate	hold-out test risk	hold-out certificate
spambase	3217	0.053	0.274	0.054	0.071
satimage	4504	0.064	0.255	0.066	0.078
pageblocks	3831	0.021	0.140	0.025	0.038
abalone8	2923	0.143	0.571	0.142	0.169
wdbc	398	0.058	0.432	0.062	0.105
german	700	0.248	0.998	0.247	0.340

2. The votes are practically equally accurate: withholding 30% of the training set costs at most 0.004 of test risk here (`pageblocks`: 0.021 vs 0.025), because forests saturate quickly. The certificates, however, differ by a factor 3 to 5: 0.071 against 0.274 on `spambase`, 0.038 against 0.140 on `pageblocks`, 0.340 against 0.998 on `german`. The hold-out certifies the vote *itself* with a single kl inversion and no KL term, whereas the self-bounding certificate goes through the Gibbs-to-vote relation (factor 4) and uses the smaller pairwise OOB samples. This reproduces, on the datasets of the course, the conclusions of Section 6.4 of the notes (Figure 42).
3. Self-bounding is preferable when the certificate is also used to *learn* or to *select* (Parts 2.1 to 2.4: it then comes for free, with a union bound in $\ln K$), when data are too scarce to set a test sample aside, or for Gibbs classifiers and stochastic predictors, for which the Gibbs-to-vote factor disappears (Part 4.1). For reporting a guarantee on a fixed forest with enough data, the hold-out is hard to beat.

Part 3.3 – Imbalanced datasets and the balanced error (Section 6)

The bounds of this course control the 0-1 risk. On imbalanced datasets a trivial classifier predicting the majority class has a small 0-1 risk.

1. On the imbalanced datasets, compare the certificates of the uniform forest with the risk of the trivial classifier. Are the certificates informative?
2. Obtain a certificate on the *balanced* error $\frac{1}{2}\big(\mathbb{P}(h(x)\neq y\,|\,y=+1)+\mathbb{P}(h(x)\neq y\,|\,y=-1)\big)$: apply the OOB bounds separately to the training examples of each class (with n_+ and n_- OOB examples) and use a union bound. Implement it and comment.

Solution

Construction. Conditionally on $y=+1$, the examples of the positive class are i.i.d. from $\mathcal{D}_{|y=+1}$, and the OOB construction applies to them: restricting the OOB masks to the positive training examples gives OOB losses and tandem losses whose expectations are the class-conditional error rates. We compute the FO and TND certificates on each class with $\delta/4$ (2 classes $\times$ 2 bounds), keep the smallest on each class, and average them: with probability $\geq 1-\delta$ both class-conditional certificates hold, hence their average bounds the balanced error.

Algorithm 11: Certificate on the balanced error

for $c\in\{+1,-1\}$ **do**
  $I_c\leftarrow\{i : y_i=c\}$; restrict every OOB set to I_c: $T_t^c\leftarrow T_t\cap I_c$;
  compute $\widehat{L}^c$, $\widehat{\mathbf{L}}^c$, n_1^c, n_2^c on these restricted sets;
  $\mathcal{B}_c\leftarrow\min\big(\mathrm{FO}(\delta/4),\mathrm{TND}(\delta/4)\big)$; ▷ *2 classes × 2 bounds*
return $\frac{1}{2}(\mathcal{B}_{+1}+\mathcal{B}_{-1})$; ▷ *valid with probability* $\geq 1-\delta$

Python: function `class_restricted, part7` of `tp_solutions.py`.

	trivial	vote	0-1 cert.	positives: error	cert.	balanced error	balanced cert.
abalone8	0.136	0.141	0.505	0.931	1.000	0.473	0.630
pageblocks	0.102	0.021	0.125	0.110	0.680	0.060	0.380
satimage	0.097	0.062	0.250	0.537	1.000	0.274	0.565
segmentation	0.143	0.027	0.214	0.143	0.837	0.076	0.492
yeast3	0.110	0.046	0.279	0.265	1.000	0.142	0.594

1. The trivial classifier has a 0-1 risk equal to the proportion of positives (0.10 to 0.14). The 0-1 certificates of the forest are larger than this value on every dataset, even on `pageblocks` (0.125 vs 0.102) where the forest is five times better than the trivial classifier on the test set. In other words, the 0-1 certificates cannot even guarantee that the forest is better than predicting the majority class.

2. The balanced error reveals what the 0-1 risk hides: on `abalone8` the vote misses 93% of the positive examples and its balanced error (0.47) is that of a random guess, although its 0-1 risk is 0.141. The balanced certificates are loose, because the positive class contains only a few hundred examples (0.68 on `pageblocks`, vacuous on three datasets), but they are honest: they certify what matters for an imbalanced problem, and they show that informative guarantees on the minority class require many minority examples. A natural improvement is to rebalance the bootstrap samples so that the trees see more positives.

Part 3.4 – Boosting (Section 7.3)

AdaBoost also outputs a weighted majority vote. To obtain a certificate, the voters must be evaluated on examples independent of them.

1. Split the training set into two halves S_1, S_2. Run `AdaBoostClassifier` with decision stumps (100 rounds) on S_1, and consider the learned stumps as the voters, with the uniform prior.
2. Compute on S_2 the FO and TND certificates of the posterior $\rho_t \propto \alpha_t$ given by AdaBoost (with $\delta/2$ each), and of the posteriors minimizing the two bounds. Compare the test risks with those of AdaBoost trained on the whole training set.
3. Why are the certificates of the AdaBoost posterior loose, although AdaBoost is accurate? *Hint:* compute the Gibbs risk of the stumps on S_2 and the KL of the AdaBoost posterior.

Solution

Algorithm 12: Certifying AdaBoost with an independent half

split the training set into S_1, S_2 ($n = |S_2|$);
run AdaBoost on S_1: stumps $h_1, \dots, h_T$ and weights α_t; $\pi \leftarrow$ uniform; $\rho_{\text{Ada}} \leftarrow \alpha / \sum_t \alpha_t$;
on S_2: $\widehat{L}_t \leftarrow$ error of h_t; $\widehat{L}_{tt'} \leftarrow$ joint errors of $h_t, h_{t'}$;
certificates of ρ_{Ada}: FO and TND with $\delta/2$ each;
$\rho_{\text{FO}}, \rho_{\text{TND}} \leftarrow$ `optimize_fo`, `optimize_tnd` on these statistics; their certificates;

Python: function `part8` of `tp_solutions.py`.

dataset	AdaBoost (all data)	AdaBoost (S_1)	AdaBoost posterior on S_2 Gibbs	KL	FO cert.	TND cert.	FO-optimized test	FO-optimized FO cert.	TND-optimized test	TND-optimized TND cert.
australian	0.148	0.150	0.430	0.30	1.000	1.000	0.147	0.541	0.147	0.908
german	0.245	0.242	0.453	0.39	1.000	1.000	0.301	0.782	0.287	1.000
pima	0.257	0.263	0.443	0.29	1.000	1.000	0.268	0.789	0.247	1.000
spambase	0.064	0.065	0.399	0.29	0.900	0.821	0.204	0.526	0.104	0.561
wdbc	0.043	0.043	0.265	0.03	0.783	0.709	0.103	0.332	0.063	0.465

2. Training on half of the data costs AdaBoost almost nothing (0.148 → 0.150 on `australian`).

The certificates of its posterior, however, are vacuous on three datasets and loose on the other two (0.71 on `wdbc`). Reweighting the stumps by minimizing the bounds improves the certificates markedly (TND certificate 0.54 on `australian`, 0.33 on `wdbc`), but mostly at the expense of the vote: the TND-optimized vote errs on 20% of `spambase` instead of 6.5%, and on 10% of `wdbc` instead of 4.3%. The optimized posteriors concentrate on the few stumps that are good on their own, and lose the combination built by AdaBoost.

3. The explanation lies in the nature of boosting. Its voters are *weak* on purpose: the Gibbs risk of the stumps on S_2 is 0.40–0.45 on four datasets out of five (0.27 on `wdbc`). AdaBoost makes them accurate *together*, by putting them in a precise combination with large margins, not by making each of them good. The first-order bound is then $2 \times 0.43 > 0.8$, and the tandem bound, which only uses the joint errors of pairs, cannot see a combination that relies on many voters at once. The KL of the AdaBoost posterior is small (0.03 to 0.4 nats), so the complexity is not the problem: the looseness comes from the Gibbs-to-vote relation. This is why boosted votes are better analysed with margin bounds (Section 5.9), or certified with a hold-out sample (Part 3.2).